

Guia para Colaboradores

Modelacion Estadistica

Maestria en Optimizacion y Modelacion de Procesos

Centro de Investigacion en Matematicas, A.C.

Unidad Aguascalientes — CIMAT

Repositorio: github.com/mop-cimat-ags/NotasModelacionEstadistica

Libro en linea: mop-cimat-ags.github.io/NotasModelacionEstadistica

Guía de referencia para catedráticos que contribuyen al libro

Modelación Estadística de la Maestría en Optimización y Modelación de Procesos (OMP), CIMAT Aguascalientes.

Repositorio: <https://github.com/mop-cimat-ags/NotasModelacionEstadistica>

Libro en línea: <https://mop-cimat-ags.github.io/NotasModelacionEstadistica>

Catedráticos colaboradores

Nombre	Correo
Dr. Sergio M. Nava Muñoz	nava@cimat.mx
Dr. Magin Zúñiga Estrada	magin.zuniga@cimat.mx
Dr. Humberto Martínez Bautista	humberto.martinez@cimat.mx

Compilación y edición del repositorio: Vladimir Jiménez Pérez (alumno) — vladimir.jimenez@cimat.mx

Contenido

- [Herramientas necesarias](#1-herramientas-necesarias)
- [Configurar Git y GitHub desde cero](#2-configurar-git-y-github-desde-cero)
- [Clonar y abrir el proyecto](#3-clonar-y-abrir-el-proyecto)
- [Estructura del proyecto](#4-estructura-del-proyecto)
- [Renderizar localmente con Quarto](#5-renderizar-localmente-con-quarto)
- [Cómo editar un capítulo](#6-cómo-editar-un-capítulo)
- [Cómo crear un capítulo nuevo](#7-cómo-crear-un-capítulo-nuevo)
- [Sintaxis básica de Quarto](#8-sintaxis-básica-de-quarto)
- [Usar la librería modelEstCIMAT](#9-usar-la-librería-modelestcimat)
- [Agregar funciones y datos a la librería](#10-agregar-funciones-y-datos-a-la-librería)
- [Flujo de trabajo con Git](#11-flujo-de-trabajo-con-git)
- [Git en VS Code (sin terminal)](#12-git-en-vs-code-sin-terminal)
- [Trabajo con ramas (branches)](#13-trabajo-con-ramas-branches)
- [Subir cambios a GitHub](#14-subir-cambios-a-github)
- [Preguntas frecuentes y errores comunes](#15-preguntas-frecuentes-y-errores-comunes)

1. Herramientas necesarias

Instala todo lo siguiente antes de empezar. El orden importa.

Herramienta	Descarga	Para qué sirve
Git	https://git-scm.com/download/win	Control de versiones
R (≥ 4.1)	https://cran.r-project.org	Ejecutar código del libro

Herramienta	Descarga	Para qué sirve
Quarto	https://quarto.org/docs/get-started	Renderizar el libro
VS Code	https://code.visualstudio.com	Editor principal
RStudio (opcional)	https://posit.co/download/rstudio-desktop	Alternativa a VS Code para R

Nota: En Windows, al instalar Git selecciona la opción

"Git from the command line and also from 3rd-party software"

para que VS Code lo detecte automáticamente.

Extensiones de VS Code recomendadas

Abre VS Code, ve a Extensiones (**Ctrl+Shift+X**) e instala:

- **Quarto** — soporte para archivos `.qmd` con previsualización
- **R** — resaltado de sintaxis y autocompletado para R
- **GitLens** — visualización avanzada de cambios y historial Git
- **GitHub Pull Requests** — revisar y crear Pull Requests desde VS Code

Paquetes de R necesarios

Abre R o RStudio y ejecuta una sola vez:

```
install.packages(c("devtools", "knitr", "rmarkdown", "here"),
                 repos = "https://cloud.r-project.org")
```

2. Configurar Git y GitHub desde cero

Esta sección cubre todo desde crear tu cuenta hasta autenticarte con GitHub. Solo se hace una vez.

2.1 Crear cuenta en GitHub

- Ve a <https://github.com>
- Clic en **Sign up**
- Elige un nombre de usuario (recomendado: `nombre-cimat` o similar)
- Usa tu correo institucional `@ciimat.mx`
- Verifica tu correo al terminar el registro

Después de crear tu cuenta, comparte tu **nombre de usuario** con el coordinador del repositorio para que te den acceso.

2.2 Verificar que Git está instalado

Abre la terminal de VS Code con **Ctrl+ñ** (o **Ctrl+backtick**) y escribe:

```
git --version
```

Debe mostrar algo como `git version 2.44.0`. Si no aparece, reinstala Git.

2.3 Configurar tu identidad en Git

Git necesita saber quién eres para registrar los cambios. Ejecuta:

```
git config --global user.name "Tu Nombre Completo"
git config --global user.email "tu.correo@cimat.mx"
```

Verifica que quedó bien:

```
git config --global --list
```

Debe mostrar tu nombre y correo.

2.4 Configurar el editor por defecto

Para que Git use VS Code cuando necesites escribir mensajes:

```
git config --global core.editor "code --wait"
```

2.5 Autenticación con GitHub — Personal Access Token (PAT)

GitHub ya **no acepta contraseñas normales** para operaciones Git.

Necesitas crear un Token de acceso personal.

Pasos para crear el PAT:

- En GitHub, clic en tu foto → **Settings**
- Baja hasta el final → **Developer settings**
- **Personal access tokens** → **Tokens (classic)**
- **Generate new token (classic)**
- En *Note* escribe: `cimat-modelacion`
- En *Expiration* elige `No expiration` o `1 year`
- Marca el permiso `repo` (control total de repositorios privados)
- Clic en **Generate token**
- **Copia el token ahora** — solo se muestra una vez

El token tiene este aspecto: `ghp_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX`

Guardar el token para no escribirlo cada vez:

```
git config --global credential.helper store
```

La primera vez que hagas `git push`, Git te pedirá usuario y contraseña.

Escribe tu **usuario de GitHub** y pega el **token** como contraseña.

Git lo guardará automáticamente para las siguientes veces.

2.6 Alternativa: autenticación por SSH (avanzado)

SSH es más seguro y cómodo que el PAT para uso frecuente.

Generar clave SSH:

```
ssh-keygen -t ed25519 -C "tu.correo@cimat.mx"
```

Presiona Enter en todo (acepta la ruta y deja la contraseña vacía).

Agregar la clave pública a GitHub:

```
# Muestra tu clave publica (cópiala completa)

cat ~/.ssh/id_ed25519.pub
```

- En GitHub → **Settings** → **SSH and GPG keys**
- **New SSH key**
- Pega la clave y guarda

Verificar que funciona:

```
ssh -T git@github.com

# Respuesta esperada: Hi usuario! You've successfully authenticated...
```

A partir de aquí usa URLs SSH al clonar:

```
git clone git@github.com:mop-cimat-ags/NotasModelacionEstadistica.git
```

3. Clonar y abrir el proyecto

3.1 Clonar el repositorio

Navega a la carpeta donde quieres guardar el proyecto y ejecuta:

```
git clone https://github.com/mop-cimat-ags/NotasModelacionEstadistica.git
```

Esto descarga todo el proyecto. **Solo se hace una vez.**

3.2 Entrar a la carpeta del proyecto

```
cd NotasModelacionEstadistica
```

3.3 Abrir en VS Code

```
code .
```

O desde VS Code: **File** → **Open Folder** y selecciona la carpeta.

3.4 Generar los datasets de la librería (solo la primera vez)

```
setwd("libreria")

source("data-raw/generar_datos.R")

setwd("../")
```

4. Estructura del proyecto

```
NotasModelacionEstadistica/

■

■■■ _quarto.yml                # Configuración del libro (no modificar)
■■■ _quarto-pdf.yml           # Perfil para renderizar solo PDF
■■■ _quarto-docx.yml          # Perfil para renderizar solo Word
■■■ index.qmd                 # Página de inicio del libro
■■■ referencias.qmd           # Capítulo de bibliografía
■■■ referencias.bib           # Referencias BibTeX
■■■ estilos-cimat.css         # Estilos HTML (no modificar)
■■■ GUIA_COLABORADORES.md     # Esta guía
■■■ README.md                 # Descripción del proyecto en GitHub

■

■■■ imagenes/

■   ■■■ logo-cimat.png

■

■■■ latex/                    # Archivos para el PDF (no modificar)
■   ■■■ preamble.tex
■   ■■■ portada.tex
■   ■■■ portada-docx.md

■

■■■ capitulos/

■   ■■■ parte1/                # Parte I – Probabilidad y Distribuciones
■   ■   ■■■ cap01-intro-estadistica.qmd
■   ■   ■■■ cap02-distribuciones.qmd
```

```

■ ■ ■ ■ cap03-distribucion-conjunta.qmd

■ ■ ■ ■ parte2/                                # Parte II – Inferencia Estadística

■ ■ ■ ■ cap04-inferencia.qmd

■

■ ■ ■ ■ libreria/                               # Paquete R: modelEstCIMAT

■ ■ ■ ■ R/                                     # Funciones

■ ■ ■ ■ data/                                 # Datasets (.rda)

■ ■ ■ ■ data-raw/                             # Scripts para generar datasets

■ ■ ■ ■ tests/                               # Pruebas unitarias

■

■ ■ ■ ■ .github/workflows/

■ ■ ■ ■ publish.yml                           # CI/CD automático (no modificar)

```

Regla: edita únicamente los archivos `.qmd` dentro de `capitulos/` y los archivos de `libreria/`. El resto lo administra el coordinador.

5. Renderizar localmente con Quarto

Antes de subir cualquier cambio a GitHub, verifica que el libro se ve bien en tu computadora. Quarto tiene dos modos principales.

5.1 Vista previa en tiempo real (`quarto preview`)

Es el modo que usarás mientras escribes. Abre una ventana del navegador que se actualiza automáticamente cada vez que guardas un archivo.

```
quarto preview
```

- El libro se abre en <http://localhost:4848> (o puerto similar)
- Cada vez que guardas un `.qmd`, el capítulo se re-renderiza en segundos
- Presiona `Ctrl+C` en la terminal para detenerlo

Vista previa de un solo capítulo (más rápido):

```
quarto preview capitulos/parte1/cap02-distribuciones.qmd
```

5.2 Renderizado completo (`quarto render`)

Genera todos los formatos (HTML, PDF, Word) y los guarda en `_book/`.

Úsalo para verificar el resultado final antes de hacer push.

```
quarto render
```

El resultado queda en:

```
_book/  
■■■ index.html      ← libro HTML completo  
■■■ *.html          ← un archivo por capítulo  
■■■ Modelacion-Estadistica.pdf  
■■■ Modelacion-Estadistica.docx
```

5.3 Renderizar solo un formato

Para no esperar la compilación de PDF o Word cuando solo editas HTML:

```
quarto render --to html  # solo HTML  
quarto render --to pdf   # solo PDF  → _book/  
quarto render --to docx  # solo Word → _book/
```

O usando los perfiles del proyecto:

```
quarto render --profile pdf  # → _book-pdf/  
quarto render --profile docx # → _book-docx/
```

5.4 Renderizar un solo capítulo

```
quarto render capitulos/parte2/cap04-inferencia.qmd
```

Esto genera solo ese capítulo en `_book/`. Útil para revisar rápido sin esperar que compile todo el libro.

5.5 El sistema de caché y freeze

Quarto tiene dos mecanismos para no re-ejecutar código innecesariamente:

Cache (`*_cache/`): guarda el resultado de cada chunk de R. Si el código no cambia, no lo vuelve a ejecutar. Se activa automáticamente con `cache: true` en `_quarto.yml`.

Freeze (`_freeze/`): guarda el resultado de archivos `.qmd` enteros. Si el archivo no cambió, Quarto lo omite completamente. Se activa con `freeze: auto`.

Por eso la primera vez que renderizas puede tardar varios minutos, pero las siguientes veces es mucho más rápido.

Forzar re-ejecución (cuando necesitas actualizar resultados):

```
quarto render --no-cache      # ignora el cache
quarto render --cache-refresh # actualiza el cache
```

O borra manualmente la carpeta `_freeze/` para re-ejecutar todo.

5.6 Ver el resultado antes de subir

Una vez renderizado, abre `_book/index.html` directamente en el navegador para ver el libro exactamente como quedará publicado.

En VS Code puedes hacer clic derecho sobre `_book/index.html` →

Open with Live Server (si tienes la extensión Live Server instalada).

6. Cómo editar un capítulo

6.1 Antes de empezar — trae los cambios más recientes

```
git pull
```

Hazlo **siempre** antes de empezar a editar para evitar conflictos.

6.2 Abre el archivo .qmd correspondiente

En VS Code, navega en el panel lateral a `capitulos/` y abre el archivo.

Por ejemplo: `capitulos/parte1/cap02-distribuciones.qmd`

6.3 Escribe o edita el contenido

Usa la sintaxis de Quarto (ver sección 8). Mientras escribes, activa la vista previa para ver los cambios al instante:

```
quarto preview
```

6.4 Verifica el resultado localmente

```
quarto render capitulos/parte1/cap02-distribuciones.qmd
```

Revisa que no haya errores en la terminal y que el capítulo se ve bien.

6.5 Sube los cambios

```
git add capitulos/parte1/cap02-distribuciones.qmd
git commit -m "actualizo ejemplos de distribucion normal en cap02"
```

```
git push
```

7. Cómo crear un capítulo nuevo

7.1 Crea el archivo .qmd

Dentro de la carpeta de la parte correspondiente:

```
capitulos/parte2/cap05-regresion-lineal.qmd
```

Plantilla mínima para empezar:

```
---  
  
title: "Regresión Lineal Simple"  
  
---  
  
## Introducción  
  
Texto de la introducción...  
  
## Modelo  
  
$$  
  

$$Y = \beta_0 + \beta_1 X + \epsilon$$
  
  
$$
```

7.2 Regístralo en _quarto.yml

Avisa al coordinador para que agregue el capítulo al índice:

```
- part: "Parte II - Inferencia Estadística"  
  
  chapters:  
    - capitulos/parte2/cap04-inferencia.qmd  
    - capitulos/parte2/cap05-regresion-lineal.qmd # nuevo
```

7.3 Convención de nombres de archivos

Usa siempre este formato: `capNN-nombre-descriptivo.qmd`

```
cap05-regresion-lineal.qmd    ✓ correcto
```

cap05_RegresionLineal.qmd	✗ evitar mayúsculas y guiones bajos
Cap 05 Regresion.qmd	✗ evitar espacios

8. Sintaxis básica de Quarto

Encabezados

```
## Sección principal

### Subsección

#### Sub-subsección
```

No uses # (título de nivel 1) — Quarto ya lo toma del `title:` en el YAML.

Texto con formato

```
**negrita**   *cursiva*   `codigo en línea`

> Esto es una cita o nota destacada.
```

Listas

```
- elemento 1

- elemento 2
  - subelemento

1. primer paso

2. segundo paso
```

Código R

```
x <- c(1, 2, 3, 4, 5)
mean(x)
```

Opciones útiles de chunk:

```
#| echo: false # oculta el código, muestra solo el resultado
#| eval: false # muestra el código pero NO lo ejecuta
#| include: false # ejecuta pero no muestra nada (setup silencioso)
#| warning: false # oculta advertencias
```

```
#| fig-cap: "Mi figura"
#| fig-width: 7
#| fig-height: 4
plot(x)
```

Figuras

```
![Descripción de la imagen](imagenes/mi-imagen.png){width=70%}
```

Ecuaciones

Ecuación en línea: $\mu = \frac{1}{n} \sum_{i=1}^n x_i$

Ecuación centrada:

$$f(x) = \frac{1}{\sigma \sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

$$f(x) = \frac{1}{\sigma \sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

$$f(x) = \frac{1}{\sigma \sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Tablas

Variable	Media	Desv. Est.
Altura	168 cm	8 cm
Peso	68 kg	12 kg

Teoremas, definiciones y ejemplos

```
::: {#thm-grandes-numeros}
```

```
## Ley de los Grandes Números
```

Si $X_1, X_2, \dots$ son iid con media μ , entonces

$\bar{X}_n \xrightarrow{p} \mu$.

```
:::
```

```
::: {#def-esperanza}
```

```
## Esperanza Matemática
```

La esperanza de X se define como $E[X] = \int x f(x) dx$.

```
:::
```

Referencias cruzadas

Como establece el @thm-grandes-numeros, la media converge...
Ver la @def-esperanza para la definición formal.

Notas al pie

El estimador es consistente.^[^1]

[^1]: Un estimador es consistente si converge en probabilidad al verdadero parámetro.

Citas bibliográficas

Agrega la referencia en `referencias.bib` y cítala:

Como demuestra @casella2002statistical [p. 234]...

Formato BibTeX en `referencias.bib`:

```
@book{casella2002statistical,  
  title    = {Statistical Inference},  
  author   = {Casella, George and Berger, Roger L.},  
  year     = {2002},  
  publisher = {Duxbury},  
  edition  = {2nd}  
}
```

Callouts (cajas destacadas)

```
::: {.callout-note}  
Esto es una nota informativa.  
:::  
  
::: {.callout-warning}  
Esto es una advertencia importante.  
:::  
  
::: {.callout-tip}
```

Esto es un consejo o truco útil.

:::

9. Usar la librería modelEstCIMAT

Al inicio de cada capítulo que use funciones de la librería, agrega este chunk de configuración:

```
#| include: false
```

```
devtools::load_all(here::here("libreria"), quiet = TRUE)
```

Funciones disponibles

```
# Estadística descriptiva

resumen_estadistico(x)

tabla_frecuencias(x, k = 6)

curtosis(x)

asimetria(x)

momento_central(x, r = 3)

coef_variacion(x)

# Inferencia

ic_media(x, nivel = 0.95)

ic_media(x, nivel = 0.95, sigma = 2)  # varianza conocida

ic_proporcion(x = 30, n = 100)

ic_varianza(x)

prueba_z(x, mu0 = 0, sigma = 1, alternativa = "bilateral")

emv_normal(x)

emv_exponencial(x)

emv_poisson(x)

# Simulación

simular_distribucion("normal", n = 500, mean = 0, sd = 1, seed = 1)

simular_bootstrap(x, FUN = mean, B = 1000, seed = 1)
```

```
simular_ic_bootstrap(x, FUN = median, B = 2000, metodo = "bca")

monte_carlo_integral(function(x) x^2, a = 0, b = 1, N = 100000)

# Gráficas

graficar_densidad("t", df = 10, cola = "bilateral", alpha = 0.05)

graficar_densidad("normal", mean = 0, sd = 1, a = -1.96, b = 1.96)

graficar_ic(ic_media(x))
```

Datasets disponibles

```
data(alturas_estudiantes) # n=80, altura/peso de estudiantes

data(tiempos_falla)       # n=60, tiempos exponenciales (fallas)

data(calificaciones_curso) # n=35, tres exámenes correlacionados

data(conteos_defectos)    # n=50, conteos Poisson (manufactura)

data(temperaturas_mensual) # n=365, temperaturas diarias Guanajuato
```

10. Agregar funciones y datos a la librería

10.1 Agregar una función nueva

Paso 1 — Abre o crea un archivo en `libreria/R/`:

- `estadisticas.R` — estadística descriptiva
- `inferencia.R` — IC, pruebas, EMV
- `simulacion.R` — bootstrap, Monte Carlo

O crea uno nuevo, ej. `libreria/R/regresion.R`.

Paso 2 — Escribe la función con documentación roxygen2:

```
#' Título breve de la función
#'
#' Descripción más detallada.
#'
#' @param x Vector numérico de entrada.
#' @param nivel Nivel de confianza entre 0 y 1 (default: 0.95).
#' @return Descripción de lo que devuelve.
```

```
#' @examples

#' mi_funcion(c(1, 2, 3, 4, 5))

#' @export

mi_funcion <- function(x, nivel = 0.95) {

  mean(x) * nivel

}
```

La línea `@export` es obligatoria para que la función quede disponible.

Paso 3 — Agrega el nombre al `libreria/NAMESPACE`:

```
export(mi_funcion)
```

Paso 4 — Recarga y prueba:

```
devtools::load_all("libreria")

mi_funcion(c(10, 20, 30))
```

10.2 Agregar datos desde CSV

- Copia tu archivo a `libreria/data-raw/mis_datos.csv`
- Agrega al final de `libreria/data-raw/generar_datos.R`:

```
mis_datos <- read.csv("data-raw/mis_datos.csv")

save(mis_datos, file = "data/mis_datos.rda")
```

- Ejecuta el script:

```
setwd("libreria")

source("data-raw/generar_datos.R")

setwd("../")
```

10.3 Agregar datos desde Excel

```
install.packages("readxl", repos = "https://cloud.r-project.org")

# En generar_datos.R:

mis_datos <- readxl::read_excel("data-raw/mis_datos.xlsx")

save(mis_datos, file = "data/mis_datos.rda")
```

10.4 Documentar el dataset

Agrega en `libreria/R/datos.R`:

```
#' Nombre descriptivo del dataset
#
#' @format Data frame con N filas y M columnas:
#' \describe{
#'   \item{columna1}{Descripción.}
#'   \item{columna2}{Descripción.}
#' }
#' @source Origen de los datos.
#' @examples
#' data(mis_datos)
#' head(mis_datos)
"mis_datos"
```

11. Flujo de trabajo con Git

Git registra todos los cambios del proyecto como fotografías (commits).

Si algo sale mal, siempre puedes regresar a una versión anterior.

Comandos esenciales

Comando	Qué hace
<code>git pull</code>	Trae los cambios más recientes de GitHub
<code>git status</code>	Muestra qué archivos modificaste
<code>git diff archivo</code>	Muestra exactamente qué cambió en un archivo
<code>git add archivo.qmd</code>	Marca un archivo para incluir en el commit
<code>git add .</code>	Marca todos los archivos modificados
<code>git commit -m "mensaje"</code>	Guarda los cambios con descripción
<code>git push</code>	Sube los cambios a GitHub
<code>git log --oneline</code>	Historial de commits resumido
<code>git restore archivo</code>	Descarta cambios no guardados en un archivo

Mensajes de commit

Escribe mensajes claros en español que describan qué cambiaste:

```
# Buenos mensajes

git commit -m "agrego seccion de distribuciones discretas en cap02"

git commit -m "corrijo formula de varianza muestral en cap01"

git commit -m "agrego dataset de precios de vivienda en libreria"

# Malos mensajes (evitar)

git commit -m "cambios"

git commit -m "fix"

git commit -m "update"
```

Ver el historial de cambios

```
git log --oneline          # resumen compacto

git log --oneline -10      # últimos 10 commits

git log --oneline --graph  # con gráfico de ramas
```

Deshacer cambios

```
# Descartar cambios no guardados en un archivo

git restore capitulos/parte1/cap02-distribuciones.qmd

# Descartar TODOS los cambios no guardados (cuidado)

git restore .

# Sacar un archivo del área de staging (después de git add)

git restore --staged archivo.qmd
```

12. Git en VS Code (sin terminal)

VS Code tiene una interfaz gráfica completa para Git. Si prefieres no usar la terminal, puedes hacer todo desde aquí.

Abrir el panel de control de código fuente

Clic en el ícono de rama en la barra lateral izquierda (o **Ctrl+Shift+G**).

Ver cambios

Los archivos modificados aparecen en la sección **Changes**.

Haz clic en cualquier archivo para ver exactamente qué líneas cambiaron (verde = agregado, rojo = eliminado).

Hacer commit

- En el panel Source Control, escribe el mensaje en el campo de texto
- Haz clic en el ícono ✓ (**Commit**)
- O presiona **Ctrl+Enter**

Push y Pull

En la barra inferior de VS Code aparece un ícono con flechas:

-  1 — tienes 1 commit listo para subir → clic para hacer push
-  1 — hay 1 commit nuevo en GitHub → clic para hacer pull

También puedes ir al menú ... en el panel Source Control y elegir

Push o **Pull**.

Trabajar con ramas en VS Code

- La rama actual aparece en la esquina inferior izquierda
- Clic en ella para cambiar de rama o crear una nueva
- Para crear: escribe el nombre y selecciona **Create new branch**

Resolver conflictos en VS Code

Cuando hay un conflicto, VS Code lo muestra con botones de ayuda:

- **Accept Current Change** — queda tu versión
- **Accept Incoming Change** — queda la versión de GitHub
- **Accept Both Changes** — combina las dos versiones
- **Compare Changes** — muestra las diferencias lado a lado

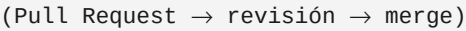
13. Trabajo con ramas (branches)

Una rama es una copia independiente del proyecto donde puedes trabajar sin afectar la versión principal (**main**).

¿Por qué usar ramas?

- Evitas sobrescribir el trabajo de otros colaboradores
- Tus cambios se revisan antes de integrarse al libro publicado
- Si algo sale mal, **main** sigue intacto

Flujo recomendado



```

git commit -m "agrego introduccion y modelo de regresion lineal"

git add .

git commit -m "agrego ejemplos con dataset alturas_estudiantes"

# 5. Subir la rama a GitHub

git push origin cap05-regresion-lineal

# 6. Abrir Pull Request en GitHub

#   → aparece un aviso amarillo en el repo

#   → clic en "Compare & pull request"

#   → describe los cambios y clic en "Create pull request"

#   → el coordinador revisa y hace merge

# 7. Después del merge, limpiar

git checkout main

git pull

git branch -d cap05-regresion-lineal

```

Guardar trabajo sin hacer commit (stash)

Si necesitas cambiar de rama pero no quieres hacer commit todavía:

```

git stash          # guarda temporalmente tus cambios

git checkout main  # cambia de rama

# ... haz lo que necesites ...

git checkout cap05-regresion-lineal

git stash pop      # recupera tus cambios

```

14. Subir cambios a GitHub

Flujo estándar

```

git pull          # 1. Trae últimos cambios

git status        # 2. Revisa qué modificaste

```

```
git add . # 3. Marca los archivos
git commit -m "descripcion clara" # 4. Guarda con mensaje
git push # 5. Sube a GitHub
```

Al hacer push, GitHub Actions compilará y publicará el libro automáticamente en **2-3 minutos**.

Puedes ver el progreso en:

<https://github.com/mop-cimat-ags/NotasModelacionEstadistica/actions>

Si hay conflicto

Un conflicto ocurre cuando dos personas modificaron el mismo archivo.

Git te avisa con:

```
CONFLICT (content): Merge conflict in capitulos/parte1/cap02.qmd
```

Abre el archivo en VS Code — verás las secciones en conflicto:

```
<<<<<< HEAD
Tu versión del texto
=====
La versión que viene de GitHub
>>>>>> origin/main
```

Edita el archivo dejando solo la versión correcta, elimina las marcas

<<<<<<, =====, >>>>>> y luego:

```
git add capitulos/parte1/cap02.qmd
git commit -m "resuelvo conflicto en cap02"
git push
```

Si el conflicto es complejo, consulta al coordinador antes de resolverlo.

15. Preguntas frecuentes y errores comunes

¿Cómo veo el libro publicado?

En <https://mop-cimat-ags.github.io/NotasModelacionEstadistica>

¿Cuánto tarda en publicarse después de un push?

2 a 3 minutos. Progreso en:

<https://github.com/mop-cimat-ags/NotasModelacionEstadistica/actions>

¿Puedo renderizar solo un capítulo localmente?

Sí:

```
quarto render capitulos/parte1/cap02-distribuciones.qmd
```

¿Por qué `quarto preview` tarda tanto la primera vez?

Porque ejecuta todo el código R y construye el caché. Las siguientes veces será mucho más rápido gracias al sistema de freeze y caché.

¿Por qué me pide usuario y contraseña al hacer push?

GitHub no acepta contraseñas. Usa tu **Personal Access Token** (ver sección 2.5) como contraseña. Si ya lo configuraste antes y te vuelve a pedir, ejecuta `git config --global credential.helper store` y vuelve a intentar el push.

Error: `remote: Repository not found`

No tienes acceso al repositorio. Contacta al coordinador para que te agregue como colaborador en GitHub.

Error: `error: failed to push some refs`

Alguien más hizo push antes que tú. Primero haz `git pull`, resuelve conflictos si los hay, y luego `git push`.

Error: `quarto: command not found`

Quarto no está en el PATH. Reinstala Quarto y reinicia VS Code.

¿Puedo editar directamente en GitHub sin clonar?

Sí, para cambios pequeños: abre el archivo en GitHub → ícono del lápiz (Edit). No recomendado para cambios grandes ni para agregar imágenes.

¿Cómo agrego una imagen al capítulo?

- Guarda la imagen en `imagenes/` o junto al capítulo
- Refiérela en el `.qmd`:

```
![Descripcion de la imagen](imagenes/mi-imagen.png){width=70%}
```

¿Qué hago si rompí algo y quiero volver atrás?

Si aún no hiciste commit:

```
git restore nombre-del-archivo.qmd # descarta cambios en ese archivo
git restore .                       # descarta TODOS los cambios
```

Si ya hiciste commit, consulta al coordinador — hay formas de revertir pero es mejor hacerlo con ayuda.

¿Qué es `_freeze/` y puedo borrarlo?

Es el caché de Quarto que guarda los resultados de los chunks. Si lo borras, Quarto volverá a ejecutar todo el código la próxima vez que renderices (tarda más). No es necesario borrarlo a menos que quieras forzar una actualización